

基本框架与 cout

主函数

主函数表明了 C++ 程序的主体，入口。主函数 每个 c++ 程序有且仅有一个

```
int main(){           // int--Integer-- 返回值是整数类型
    // {} 中间是函数的执行语句 c++ 程序运行后从 main 函数内的第一行开始执行
    return 0; // return-- 返回的结果 这里是整数 0 (通常返回 0 表示程序正常结束)
}
```

基本框架

C++ 基本框架指的是包含输入输出库的源代码，它可以完成基本的输入输出工作。

```
#include<iostream>    // #include 包含一个头文件 iostream 标准输入输出流
using namespace std;  // 使用命名空间 std; standard 的缩写, 标准的
int main(){           // std 是使用 cin, cout, endl 所必备的
    // 程序主体
    return 0;
}
```

cout 语句

cout 即 C++ out 语句，是用来输出信息的，流式输出

```
cout<<"Hello World!"; cout << "中文"; // 输出一句话、中文需放到双引号中输出
cout<<21; cout << 6+15;                // 输出数字可直接输出，算式输出结果
cout<<'a';                             // 输出字符(数字、字母、符号)需放单引号中输出
cout<<endl; // 输出一个回车             // endl 的缩写，是回车键
cout<<1<<"+"<<1<<"="<<1+1<<endl; // 用<< 隔开不同的输出 流式输出
```

C++ 注释

注释是程序的解释性语句，它可以提高源代码的可读性

C++ 支持单行注释和多行注释。注释中的所有内容会被 C++ 编译器忽略而不执行

C++ 注释一般有两种：

```
//          用于单行注释
/* ... */    用于多行注释
```

部分符号的输出方式

需要使用反斜杠来输出一些特殊含义的符号

双引号：\" 单引号：\' 反斜杠：\\ 换行：\n

练习

请写出以下 cout 语句会输出什么内容（方框中写下内容，注意符号，是否换行）

1. cout 用双引号输出一句话，中英文均可，默认不换行

```
cout << "Hello World!";  
cout << "你好，世界";
```

2. endl 换行和\n 换行

```
cout << "Hello World!"<<endl;  
cout << "你好\n世界";
```

3. 输出数学算式

```
cout<<1+1<<endl;  
cout<<"1+1"<<endl;  
cout<<1<<"+"<<1<<"="<<1+1<<endl;
```

4. 输出特殊符号

```
cout<<"'"<<endl;  
cout<<"\"<<endl;  
cout<<"\"<<endl;
```

双引号中再放双引号会有歧义，所以使用\来特殊表示

5. 单引号表示一个符号，如

```
cout<<'a'<<'A'<<'0'<<'+'<<endl;
```

6. 空格在双引号外不起作用

```
cout<<  "*"<<endl;  
cout<<  "***"<<endl;  
cout<<"*****"<<endl;  
cout<<endl;  
cout<<"  *"<<endl;  
cout<<" ***"<<endl;  
cout<<"*****"<<endl;
```

请默写出基础框架

知识拓展-C 语言与 C++的关系

C 语言是一门**面向过程**的设计语言，广泛应用于底层开发。

C++是一种计算机高级程序设计语言，由 C 语言扩展升级而产生，最早于 1979 年由本贾尼·斯特劳斯特卢普在 AT&T 贝尔工作室研发。既可以进行 C 语言的**过程**化程序设计，又可以**面向对象**的程序设计。C++继承了 C 语言的所有内容，这也是为什么可以在 C++中使用 C 语言函数的原因。

变量与输入

变量的含义

变量用来存储数据的一块内存空间；
根据变量类型的不同，空间大小与结构都不同
变量可使用也可以修改



变量创建与初始化

变量创建方式：数据类型 变量名； // 不初始化则可能存储随机值

`int a;` // 创建整数型变量 `a`

变量初始化：数据类型 变量名=初始值； // 初始化就是创建变量时赋值

`int a=0;` // 创建 `a` 并存储 `0`

赋值语句：变量名=变量值 变量 1=变量 2

`a=9;` // 将 `a` 的值修改为 `9`

`a=4*5;` // 将 `a` 的值修改为 `4*5` 的结果

`b=a;` // 将 `b` 的值修改为变量 `a` 的值

多个变量一次创建：数据类型 变量 1，变量 2，变量 3； // 创建多个变量中间用逗号，隔开

`int a=0,b=1,c=2,d;` // 四个变量都是整数类型

- 赋值是从右到左看的，`b=a;` // `a` 的值赋值给 `b`
- 赋值是覆盖等号左边的变量，`b=a;` // 不管 `b` 原来存的什么，都会被覆盖
- 赋值是复制右侧的变量给左侧，`b=a;` // `a` 的值不会改变
- 赋值类型不同时，`int a=3.99;` // `a` 会被赋值为 `3`，小数会被舍去

变量的命名规则

在定义变量的时候要给变量起名字，变量的名字是一种“标识符”，意思就是用来识别各种不同的变量，同理函数名等也要遵循以下规则

1. 标识符只能由字母、数字和下划线（`_`）组成
2. 数字不可以出现在第一个位置（比如 `1a` 不行）
3. 标识符不可以用 C++ 的关键词（保留字，如 `int`、`using` 等）
4. 区分大小写（`asd` 和 `Asd` 是不同的两个变量）

cin 语句

`cin` 可以将命令行中的输入读取到变量中，其格式一般为：

`cin>>变量名;`

`cin>>a;` // `a` 要先创建

`cin>>a>>b;` // 输入多个变量

常量

常量是固定值，在程序执行期间不会改变。如 `int a=134;` //这里的数字134 就是常量

```
cout<<011; //0 开头表示八进制 输出9
cout<<0x11; //0x 开头表示十六进制 输出17
cout<<3.14E2; //科学计数法，表示3.14*10*10，输出314
cout<<'\n'<<'a'; //字符常量
cout<<"hello world 你好"; //字符串常量
```

定义常量

对于一些常用的常量我们可以给他取名字，之后用它更方便，比如 π 。那么这时就可以定义出常量，可以使用它，但不能修改它

1. const 修饰

在定义变量时候在前面加上 `const` 就可以把变量变成常量

```
const int CIRCLE_R=5; //创建一个常量表示圆的半径
```

常量一定在创建的时候就赋值，之后不可以再修改

2. #define 宏常量

```
#define PI 3.14159; //定义一个常量 PI 圆周率
```

在主函数上面定义常量

使用时这样写 `cout<<2*PI*CIRCLE_R;` //计算圆的周长 $2\pi R$

练习

1. 写出输出的值，注意 `cin` 也会覆盖原有值

```
int a=1,b=2;
cin>>a>>b; //输入为5 10
cout<<a+b;
```

2. 完善程序-现在要做一个程序，计算输入的两个数的乘积，如输入 4 5 输出 20

```
int a,b;

//写出这行的代码

cout<<a*b;
```

3. 写出程序的输出结果

```
int a=5;
a=a+1; //先看右再看左
cout<<a;
```

4. 现有两个变量，分别存储 5 和 10，请你完善程序使变量的值交换过来，输出 10 5

```
//引入头文件
//使用命名空间std
```

```
int main(){
    int a=5,b=10,t; //t 是临时变量
    //在下面空白处写代码使 a, b 的值交换
```

```
cout<<a<<b; //一般return 0; 可以省略不写
```

```
}
```

数据类型

位，字节

位 (bit、比特) 用小写 b 表示 内存的最小单位是“位” 一个位可以表示 0, 1 两种值
字节 (Byte) 用大写的 B 表示 内存的基本单位是“字节” 一字节等于 8 位 1B==8b
x 个位可以表示 2 的 x 次方 种信息
1 位可以表示 2 种信息 0, 1
2 位可以表示 4 种信息 00, 01, 10, 11
3 位可以表示 8 种信息 000, 001, 010, 011, 100, 101, 110, 111
.....
32 位可以表示 2 的 32 次方 4294967296 种信息

整型 int 、 long long

1. int 4 个字节 32 位 -2147483648 到 2147483647 (10 位)
可以表示负数 (2147483648 个), 0 (1 个), 正数 (2147483647 个) 加起来等于 2 的 32 次方 4294967296
2. long long 8 个字节 64 位 -9223372036854775808 到 9223372036854775807 (19 位)
long long 是 long long int 的缩写
整型变量超过最大值后会发生溢出

```
int a=2147483647;  
a=a+1;//a 增加 1  
cout<<a;//输出-2147483648 从山顶跌落到谷底  
当 int 表示不了时, 要用 long long
```



当心跌落

浮点型 float、double

浮点型可以表示极大的数字, 3.4×10^{38} , 但是受到精度的限制

1. 单精度浮点型 float 精度大概 6, 7 位

```
float a=1.123456789;  
printf("%.10f",a); //输出 1.1234568357
```
 2. 双精度浮点数 double 精度大概 15 位左右

```
double a=1.1234567890123456789;  
printf("%.20f",a); //1.12345678901234569125
```

建议直接用 double, 所有小数变量都用 double
- cout 小数时位数问题

```
cout<<3.123456798<<endl; //输出 3.12346
```

当小数超过 5 位时, 默认保留 5 位 (不同运行环境设置不同)

➤ printf

```
printf("int:%d char:%c double:%f\n",1,'a',3.14);  
printf("左侧%.5.2f 右侧\n",3.149); //输出 左侧 3.15 右侧  
//%.5.2f 表示最少占 5 个位置 小数点后保留两位小数
```

字符型 char

字符型占 1 字节 8 位，存储的是数字(ASCII 码)，表示的是符号 'A'=65 'B'=66.....
'a'=97 'b'=98 '0'=48 '1'=49.....

char 本质还是个数字，只是在输出的时候会被转换成符号

➤ 赋值方法

```
char a=65; //直接用ASCII 赋值
```

```
char b='A'; //用字符常量赋值
```

➤ 输入时，一个变量读取一个符号

```
char a,b;
```

```
cin>>a>>b; //输入 123 按回车
```

```
cout<<a<<" "<<b; //输出 1 2
```

➤ 输出时

```
char a=65;
```

```
cout<<a<<endl; //输出 A 直接输出时，输出的是字符
```

```
cout<<a+1<<endl; //输出 66 参与运算后的结果是整数类型
```

```
cout<<char(a+1)<<endl; //输出 B 用到了强制类型转换
```

```
cout<<int('D')<<endl; //输出字符D 的ASCII 码 68
```

➤ 大小写转换 大写加 32 转小写 小写减 32 转大写

```
char a='B';
```

```
a=a+32; //大写加 32 转小写
```

```
cout<<a<<endl; //输出 b
```

```
cout<<char('C'+32)<<endl; //输出 c
```

```
cout<<char('d'-32)<<endl; //输出 D
```

➤ 数字字符 转 整数 数字字符-'0' 得到 数字

```
cout<<'0'-'0'<<endl; //输出 0，这个 0 是 int 类型的，不是字符'0'
```

```
cout<<'1'-'0'<<endl; //1
```

```
cout<<'2'-'0'<<endl; //2
```

```
cout<<'3'-'0'<<endl; //3
```

```
cout<<'9'-'0'<<endl; //9
```

布尔型 bool

指的是逻辑上的 真 假 、对 错 用单词 true、false 表示，占一个字节

➤ 用 true, false 表示真假 其实 true 就是 1, false 就是 0

```
bool a=true; //如果是 false 输出 0
```

```
cout<<a; // 输出是 1 表示真
```

➤ 通过逻辑运算得到 bool 值

```
bool a=2>1;
```

```
cout<<a; // 输出 1 因为 2>1 是真的
```

➤ 通过数字代表真假 0 即假 非零即真

```
bool a=999; // 999 不是 0 所以代表逻辑真
```

```
cout<<a<<endl; //输出 1
```

```
cout<<bool(-1)<<endl; //输出 1 因为 -1 不是 0
```

```
cout<<bool(0)<<endl; //输出 0
```

隐式类型转换

运算时某些类型会悄悄的转换以适应计算，比如 `5/2.0` 或 `int a=3.5;`

➤ 计算时 转换成其中最大的类型计算

如: `'a'/2*1.1`

①先算除法，'a'和2转换成其中最大的整型 int 类型计算，结果也是 int 型 `97/2==48`

②再算乘法，之前结果是 int 型 48，转换成 double 类型进行计算，结果为 52.8

➤ 赋值时 按照左边变量的类型进行转换

如: `int a=3.5;` // 舍去小数，赋值为 3

`double b=3;` // 提升类型，存 3.0

强制类型转换

根据程序的需要，手动进行转换的方法，格式为 转换类型(被转换值)

如 `int(3.14)` // 去掉小数

`int('a')` // 获取字符的 ASCII 码

`double(5)/2` // 获取整数除整数的完整结果

`char(65)` // 一般在 cout 中输出字符使用

ASCII 码表

ASCII 字符代码表 一

高四位 低四位		ASCII非打印控制字符										ASCII 打印字符											
		0000					0001					0010	0011	0100	0101	0110	0111						
		0					1					2	3	4	5	6	7						
		+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	ctrl	
0000	0	0	BLANK NULL	^@	NUL	空	16	▶	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p
0001	1	1	☺	^A	SOH	头标开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q
0010	2	2	☹	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s
0100	4	4	◆	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t
0101	5	5	♣	^E	ENQ	查询	21	♠	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v
0111	7	7	●	^G	BEL	震铃	23	↑↓	^W	ETB	传输块结束	39	'	55	7	71	G	87	w	103	g	119	w
1000	8	8	◻	^H	BS	退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x
1001	9	9	◯	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y
1010	A	10	◻	^J	LF	换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z
1011	B	11	♂	^K	VT	竖直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{
1100	C	12	♀	^L	FF	换页/新页	28	└	^\ FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124		
1101	D	13	♪	^M	CR	回车	29	↔	^] GS	组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	🎵	^N	SO	移出	30	▲	^_	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~
1111	F	15	☼	^O	SI	移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ

注：表中的ASCII字符可以用:ALT + “小键盘上的数字键” 输入

练习

1. [阅读程序] 写出程序运行的结果

```
int a=2147483647;
a=a+1;
long long b=2147483647;
b=b+1;
cout<<a<<endl<<b;
```

2. [阅读程序] 写出程序运行的结果

```
double a=2;
a=5/a;
cout<<a<<endl;
```

3. [阅读程序] 写出输出，并思考程序的作用是什么

```
char bai,shi,ge;
cin>>bai>>shi>>ge;//输入 123 按回车
cout<<ge<<shi<<bai;
```

以上程序的目的是：_____

4. [阅读程序] 写出输出，并思考程序的作用是什么

```
char bai,shi,ge;
cin>>bai>>shi>>ge;//输入 123 按回车
cout<< (bai-'0')+(shi-'0')+(ge-'0');
```

以上程序的目的是：_____

5. [阅读程序] 写出输出，并思考程序的作用是什么

```
char c1,c2,c3;
cin>>c1>>c2>>c3;//输入 ASD
c1=c1+32;
c2=c2+32;
c3=c3+32;
cout<<c1<<c2<<c3;
```

以上程序的目的是：_____

6. [编程题] pcbc.cc 题库 3959 题

小伟去买材料做蛋糕吃，买了 1 斤鸡蛋，每斤 5.5 元，花了 5.5 元。又买了 3 斤面粉，每斤 3.91，花了 11.7 (四舍五入保留 1 位小数)。接下来还要去买很多别的东西，每次都口算太麻烦了，希望你做个程序帮助他。输入两个数，一个是多少斤 (整数) 一个是价格 (小数)，输出金额 (四舍五入保留一位小数) `printf("%.1f", 金额);`

表达式计算（一）

表达式

表达式是一组运算符和运算数的组合，用来计算一个值，如下都是表达式

`a+1` `a*b` `'a'-'A'` `int(a)/int(b)` `a++` `pow(a,3)` `a&b` `a*(b+c)` `1+1`

运算符是指如 “+” “-” “++” “%” 等表示运算规则的符号

运算数则指参加运算的值，可能是常量也可能是变量，还可能是一个函数的返回值

算术运算符

假设变量 A 的值为 10，变量 B 的值为 20，则：

运算符	描述	实例
+	把两个操作数相加	A + B 将得到 30
-	从第一个操作数中减去第二个操作数	A - B 将得到 -10
*	把两个操作数相乘	A * B 将得到 200
/	分子除以分母	B / A 将得到 2
%	取模运算符，整除后的余数	B % A 将得到 0
++	自增运算符，整数值增加 1	A++ 将得到 11
--	自减运算符，整数值减少 1	A-- 将得到 9

除法 /

- 当除数和被除数都是整数时，除法的结果为整除，得到一个整数，小数部分直接丢弃
 - 5/2 的结果为 2，不是 2.5，也不是四舍五入的 3
- 当除数和被除数中有至少一个浮点数时，结果为浮点数
 - 5.0/2 5/2.0 5.0/2.0 5*1.0/2 double(5)/2 的结果都是 2.5
- 不能除零，除零能通过编译但是运行中会出错

取余 %

- 左右两边的运算数都必须是整数，不支持取余小数
- 取余的结果一定和被取余数的符号相同，正数取余的结果一定是正数，负数取余同理
 - 5%-2==1 -1%-2== -1 9%-3==0

自增自减运算符

- 先自增自减-----先改变变量的值，再参与运算

```
int a=5;  
cout<<(++a)+4;
```

- 答案是 10，先自增，a 变成 6 再参与运算

- 后自增自减-----先参与运算，再改变变量的值

```
int a=5;  
cout<<(a++)+4; // 输出 9  
cout<<a; // a 自增为 6
```

- 先将 a 的值 5 参与运算，运算后 a 再自增为 6

取个位，十位，百位……

- %10 可以取最后一位，%100 取后两位
- /10 可以去掉最后一位，/100 去掉后两位
- 对于一个变量，假设 `int a=123;`
- ```
int b=a/100; // 去掉后两位
int s=a/10%10; // 去一位然后取最后一位
int g=a%10; // 取最后一位
```

## 赋值运算符 简写 `a=a+1` 为 `a+=1`

| 运算符 | 描述                               | 实例                                                               |
|-----|----------------------------------|------------------------------------------------------------------|
| =   | 简单的赋值运算符，把右边操作数的值赋给左边操作数         | <code>C = A + B</code> 将把 <code>A + B</code> 的值赋给 <code>C</code> |
| +=  | 加且赋值运算符，把右边操作数加上左边操作数的结果赋值给左边操作数 | <code>C += A</code> 相当于 <code>C = C + A</code>                   |
| -=  | 减且赋值运算符，把左边操作数减去右边操作数的结果赋值给左边操作数 | <code>C -= A</code> 相当于 <code>C = C - A</code>                   |
| *=  | 乘且赋值运算符，把右边操作数乘以左边操作数的结果赋值给左边操作数 | <code>C *= A</code> 相当于 <code>C = C * A</code>                   |
| /=  | 除且赋值运算符，把左边操作数除以右边操作数的结果赋值给左边操作数 | <code>C /= A</code> 相当于 <code>C = C / A</code>                   |
| %=  | 求模且赋值运算符，求两个操作数的模赋值给左边操作数        | <code>C %= A</code> 相当于 <code>C = C % A</code>                   |
| <<= | 左移且赋值运算符                         | <code>C &lt;&lt;= 2</code> 等同于 <code>C = C &lt;&lt; 2</code>     |
| >>= | 右移且赋值运算符                         | <code>C &gt;&gt;= 2</code> 等同于 <code>C = C &gt;&gt; 2</code>     |
| &=  | 按位与且赋值运算符                        | <code>C &amp;= 2</code> 等同于 <code>C = C &amp; 2</code>           |
| ^=  | 按位异或且赋值运算符                       | <code>C ^= 2</code> 等同于 <code>C = C ^ 2</code>                   |
| =   | 按位或且赋值运算符                        | <code>C  = 2</code> 等同于 <code>C = C   2</code>                   |

## 练习

请写出以下 cout 语句会输出什么内容

```
int a=3,b=2;
cout<<a+b<<endl;
cout<<a-b<<endl;
cout<<a*b<<endl;
cout<<a/b<<endl;
cout<<a%b<<endl;
int c=5;
cout<<(++c)%b<<endl;
c=5;
cout<<(c++)%b<<endl;
c=5;
c+=b;
cout<<c<<endl;
c=5;
c*=b;
cout<<c<<endl;
c=5;
c/=b;
cout<<c<<endl;
```

有一个 4 位数，请你分别输出它的每一位数（在下面的括号中补充代码）

```
int a=3579;
cout<<"千位数是："<<()<<endl;
cout<<"百位数是："<<()<<endl;
cout<<"十位数是："<<()<<endl;
cout<<"个位数是："<<()<<endl;
```

计算两个时间相差几分钟，输入两个时间，包含小时和分钟，h 代表小时，m 代表分钟，第一个时间比第二个时间大

```
int h1,m1;//第一个时间的小时和分钟
int h2,m2;//第二个时间的小时和分钟
cin>>h1>>m1;
cin>>h2>>m2;
//在这里计算 第一个时间的总分钟数
int t1=();
//计算第二个时间的总分钟数
int t2=();
//计算相差多少分钟
int t=();
cout<<"总共相差了"<<t<<"分钟";
```

# 表达式计算（二）

## 关系运算符

假设变量 A 的值为 10，变量 B 的值为 20，则：

| 运算符 | 描述                              | 实例            |
|-----|---------------------------------|---------------|
| ==  | 检查两个操作数的值是否相等，如果相等则条件为真。        | (A == B) 不为真。 |
| !=  | 检查两个操作数的值是否相等，如果不相等则条件为真。       | (A != B) 为真。  |
| >   | 检查左操作数的值是否大于右操作数的值，如果是则条件为真。    | (A > B) 不为真。  |
| <   | 检查左操作数的值是否小于右操作数的值，如果是则条件为真。    | (A < B) 为真。   |
| >=  | 检查左操作数的值是否大于或等于右操作数的值，如果是则条件为真。 | (A >= B) 不为真。 |
| <=  | 检查左操作数的值是否小于或等于右操作数的值，如果是则条件为真。 | (A <= B) 为真。  |

## 逻辑运算符

假设变量 A 的值为 true，变量 B 的值为 false，则：

| 运算符 | 描述                                                  | 实例                |
|-----|-----------------------------------------------------|-------------------|
| &&  | 称为逻辑与运算符。如果两个操作数都 true，则条件为 true。                   | (A && B) 为 false。 |
|     | 称为逻辑或运算符。如果两个操作数中有任意一个 true，则条件为 true。              | (A    B) 为 true。  |
| !   | 称为逻辑非运算符。用来逆转操作数的逻辑状态，如果条件为 true 则逻辑非运算符将使其为 false。 | !(A && B) 为 true。 |

与运算相当于乘法，只要其中有 0，乘法的结果就是 0（false）

或运算相当于加法，只有两个数都是 0 时，结果才会是 0（false）

非相当于负号 能把正变成负 负变成正 （true false 互换）

优先级：非(!) > 与(&&) > 或(||)

## 关系运算符不可连写

比如判断一个变量的值是否在 0 到 10 之间，**不可以**这样写  $0 < a < 10$ ，如果这样写不管 a 是多少，都是 true

## 逻辑运算的惰性

与运算（&&）只要前者为 0，就不必算后面的  
或运算（||）只要前者 1，就不必算后面的

```
int i=1;
cout<<(false && i--); //i-- 不会执行 输出 0
cout<<i<<endl; //输出 1
```

```
int j=1;
cout<<(true || j--); //j-- 不会执行 输出 1
cout<<j<<endl; //输出 1
```

## 条件运算符

格式为 <表达式 1>?<表达式 2>:<表达式 3>

语义为：当<表达式 1>成立时，整体取<表达式 2>的值，否则取<表达式 3>的值

举例：  
`cout<<((2>1)?("2 是大于 1 的"):( "2 大于 1 不成立"));`  
`cout<<((a%2==0)?("a 是偶数"):( "a 是奇数"));`  
`cout<<"a 和 b 中最大的是"<<((a>b)?(a):(b));`

判断大小写字母：

```
char c='a';
cout<<(('A'<=c&&c<='Z')?("大写字母"):(""));
cout<<(('a'<=c&&c<='z')?("小写字母"):(""));
```

## 练习

如何判断 a 在 60 到 100 之间，包含 60 和 100，写出逻辑表达式即可\_\_\_\_\_

如何判断一个数 a 是奇数\_\_\_\_\_

如何判断一个字符 a 是数字字符\_\_\_\_\_

•普通闰年:能被 4 整除但不能被 100 整除的年份为普通闰年。(如 2004 年就是闰年，1999 年不是闰年);

•世纪闰年:能被 400 整除的为世纪闰年。(如 2000 年是世纪闰年，1900 年不是世纪闰年);  
请写出如何判断年份 a 是闰年

---

# 选择结构

## 三大结构

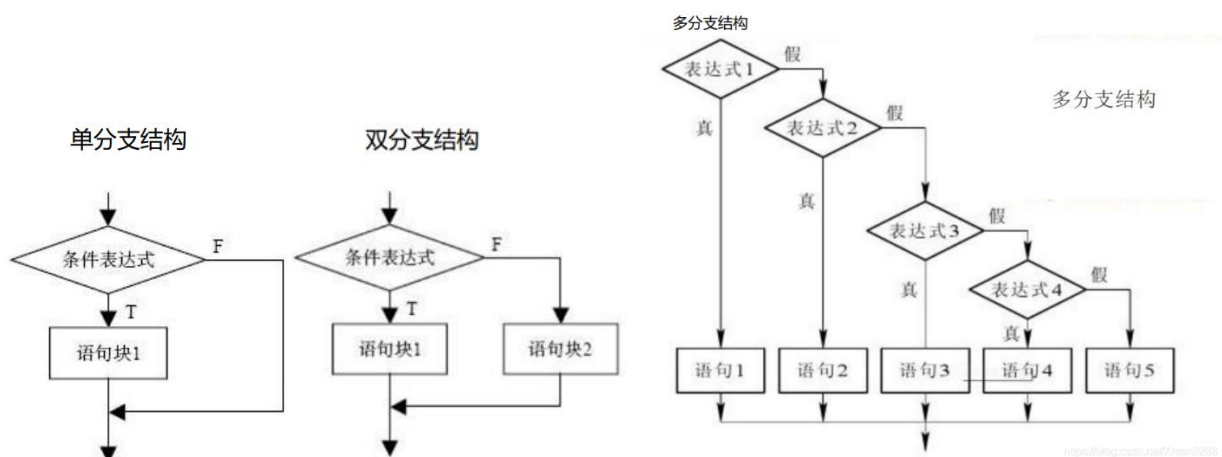
任何程序都可以由顺序，选择和循环三种基本控制结构构造。三种结构相辅相成。

**顺序结构：**执行顺序是自上而下，依次执行。

**选择结构：**判断给定的条件，根据判断的结果来控制程序的流程。

**循环结构：**条件满足，某些代码会反复多次的执行。直到条件不满足。

## 选择结构-if 语句



单分支：

```
if(条件表达式){ //如果条件满足，则执行语句块，否则不执行
 语句块
}
```

双分支：

```
if(条件表达式){ //如果条件满足，则执行语句块1，否则执行语句块2
 语句块1
}else{ //二选一，必定执行一个
 语句块2
}
```

多分支结构：

```
if(条件表达式){ //如果条件满足，则执行语句块1
 语句块1
}else if(条件表达式2){ //否则再判断条件2 是否满足
 语句块2
}else{ //以上都不满足，则执行语句块3
 语句块3
}
```

多分支只会选择其中一个分支执行

多分支中 else if 可以写多个，并且最后的 else 可以不写

分支语句 if 的写法多种多样，不同的写法也能实现同一效果。

接下来举例说明

- ✚ 除零优化 输入两个数到变量 a、b 中，输出相除的商，但除法是除 0 的明显的双分支，两种情况分别是 b 为 0 或不为 0

```
if(b==0){
 cout<<"除数不能为 0";
}else{
 cout<<a/b;
}
```

也可以写成单分支，当输入 b 为 0 时，直接结束程序

```
if(b==0){
 cout<<"除数不能为 0";
 return 0; // 结束程序
}
```

```
cout<<a/b;
```

甚至使用多分支也可以实现

```
if(b==0){
 cout<<"除数不能为 0";
}else if(b!=0){
 cout<<a/b;
}
```

三种方式各有各的特点，推荐使用跟符合逻辑的双分支

## 常用的判断举例

- ✚ 是否在某个值范围

```
if(0<a) //a 是否是负数
```

```
if(a<=5) //a 是否大于等于 5
```

```
if(110<=a&&a<150) //a 是否大于等于 110 小于 150 注意不能写成 110<=a<150
```

- ✚ 奇偶判断（整除判断）

```
if(a%2==0) //如果是偶数
```

```
if(a%2==1) if(a%2!=0) if(a%2) //如果是奇数
```

```
if(a%3==0) //如果是 3 的倍数
```

- ✚ 字符判断

```
if(a=='A') if(a==65) //char 变量的值是否是大写 A
```

```
if('A'<=a&&a<='Z') //是否是大写字符
```

```
if('a'<=a&&a<='z') //是否是小写字符
```

```
if('0'<=a&&a<='9') //是否是数字字符
```

## 简单分支举例

某小学举办了跳绳比赛，一分钟内跳绳 200 及以上的同学能被评为“跳绳达人”。

```
if(a>=200){
 cout<<"跳绳达人";
}else{
 cout<<"跳绳次数不够";
}
```

## 分支练习（一）

1. 输入一个整数，判断其奇偶性，若为奇数则输出“奇数”，若为偶数则输出“偶数”。

```
int a;
cin>>a;


```

2. 数学考试增加了一道附加题，但是总分不允许超过 100 分，老师设置了一个规则，超过一百分的设置为 100 分。未超过 100 分的任然是原来的分数。

```
int a;
cin>>a;

```

```
cout<<"最终分数是："<<a;
```

3. 判断三位数：某网站限制了用户输入只能是一个三位数（100~999），当输入正确时显示“正确”，否则显示“请重新输入”

```
int a;
cin>>a;


```

4. 数学中十进制数以 0 或 5 结尾的都能被 5 整除，现输入一个大于 0 的数，请用这一规律判断其是否能被 5 整除，输出“能”，“否”

```
int a;
cin>>a;


```

5. 判断闰年，闰年分为普通闰年和世纪闰年

- 普通闰年:能被 4 整除但不能被 100 整除的年份为普通闰年。（如 2004 年就是闰年，1999 年不是闰年）；

- 世纪闰年:能被 400 整除的为世纪闰年。（如 2000 年是世纪闰年，1900 年不是世纪闰年）；

现输入年份 a，请判断其是否是闰年，若是闰年则输出“闰年”，平年就输出“平年”

```
int a;
cin>>a;
```

## 多分支举例

- （部分条件可以省略）期末统计分数，大于等于 80 分的为优秀，大于等于 60 且小于 80 分的为良好，小于 60 分为不及格。

```
if(a>=80){
 cout<<"优秀";
}else if(a>=60){//&&a<80 可以不写
 cout<<"良好";
}else{
 cout<<"及格";
}
```

执行其中任意一种分支后就退出分支，不再继续判断。所以 `&&a<80` 是可以不写的。

- （分支顺序决定结果）以下是判断能否整除的代码。

```
if(a%2==0){
 cout<<"能被 2 整除"<<endl;
}else if(a%3==0){//&&a<80 可以不写
 cout<<"能被 3 整除"<<endl;
}else if(a%2==0&&a%3==0){
 cout<<"能被 2 和 3 同时整除"<<endl;
}
```

此时输入 `a` 为 6 会输出什么呢？怎么解决？输入一个 5 又会怎么样

- （某些分支可以合并）以下是输出两个数中最大值的代码，`a` 大则输出 `a`，相等则输出 `a` 或 `b` 都可以，`b` 大就输出 `b`

```
if(a>b){
 cout<<a;
}else if(a==b){
 cout<<a;
}else{
 cout<<b;
}
```

这样写是不是有点啰嗦了呢？你能把它优化一下吗？

- （使用 `else` 还是 `else if`）以下是一个判断输入字符类型的分支语句

```
if('0'<=a&&a<='9'){
 cout<<"是数字字符";
}else if('A'<=a&&a<='Z'){
 cout<<"是大写字母";
}else{//除开数字和大写字母以外就只有小写字母了吗
 cout<<"是小写字母";
}
```

以上的情况就属于 `else` 使用错误的情况，可以这样改

```
if....
}else if....
}else if('a'<=a&&a<='z'){//除开数字和大写字母以外就只有小写字母了吗
 cout<<"是小写字母";
}else{
 cout<<"其他符号";
}
```

## 分支练习（二）

[补全代码] 期末统计分数，大于等于 80 分的为优秀，大于等于 60 且小于 80 分的为良好，小于 60 分为不及格。（变量为 a）

```
if(){
 cout<<"优秀";
}else if(){
 cout<<"良好";
}else{
 cout<<"及格";
}
```

[四分支] 可以将字符分为大写字母，小写字母，数字字符，和其他字符。请用 if 语句完成代码。（变量为 a）

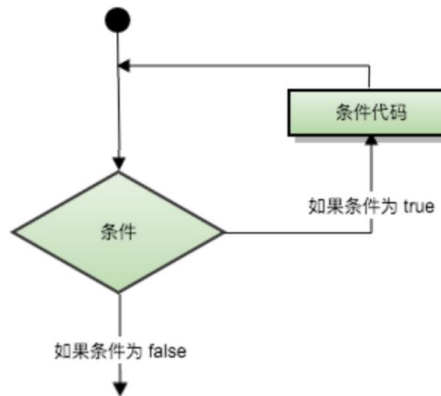
[阅读程序]

```
if(a%2==0){
 cout<<"能被 2 整除"<<endl;
}else if(a%3==0){//&&a<80 可以不写
 cout<<"能被 3 整除"<<endl;
}else if(a%2==0&&a%3==0){
 cout<<"能被 2 和 3 同时整除"<<endl;
}
```

1. 当输入变量 a 为 3 时会输出什么？
2. 当输入变量 a 为 6 时会输出什么？
3. 当输入变量 a 为 5 时会输出什么？
4. 以上代码存在哪些问题？\_\_\_\_\_和
5. 请将你认为最完善的代码写在下方

# 循环结构

## While 循环



```
while(条件){
 执行操作;
}
```

- 循环输出 1 到 10:

```
int a=1;
while(a<=10){//当a 小于10 的时候循环
 cout<<a<<endl;
 a++; //输出a 后a 增加1
}
```

- 循环累加 1 到 10:

```
int a=1,sum=0;//累加从0 开始
while(a<=10){
 sum+=a;
 a++;
}
cout<<"sum="<<sum;//输出 sum=55
```

- 循环累乘 1 到 10:

```
int a=1,sum=1;//累乘从1 开始
while(a<=10){
 sum*=a;
 a++;
}
cout<<"sum="<<sum;//输出 sum=3628800
```

- 循环输出 2 4 8 16 32 64 128 256 512 1024:

```
int a=2,sum=0;
while(a<=1024){//根据条件循环
 cout<<a<<endl;
 a*=2;
 sum++;//统计循环次数
}
cout<<sum;//总共循环10 次
```

## While 循环-拆位算法

- 倒序拆出数字的每一位:

```
int a=1234;
while(a>0){
 cout<<a%10<<" "; //使用个位
 a/=10; //去掉个位
} //输出 4 3 2 1
```

- 累加数字的每一位:

```
int a=1234, sum=0;
while(a>0){
 sum+=a%10; //使用个位
 a/=10; //去掉个位
}
cout<<sum; //输出 10
```

- 反转数字:

```
int a=1234, b=0;
while(a>0){
 b=b*10+a%10;
 a/=10;
}
cout<<b; //输出 4321
```

- 统计数字中 1 的个数:

```
int a=114123, sum=0;
while(a>0){
 if(a%10==1){
 sum++;
 }
 a/=10;
}
cout<<sum; //输出 3
```

## For 循环

- 循环输出 1 到 10:

```
for(int i=1; i<=10; i++){
 cout<<i<<endl;
}
```

- 循环累加 1 到 10:

```
int a=1,sum=0;//累加从0开始
while(a<=10){
 sum+=a;
 a++;
}
cout<<"sum="<<sum;//输出 sum=55
```

- 每次循环后增加 3:

```
for(int i=1;i<=10;i+=3){
 cout<<i<<" "; //i+=3 每次循环后i加3
} //输出 1 4 7 10
```

- 循环输出 1~n:

```
int n;
cin>>n;//n 输入多少 就循环多少次
for(int i=1;i<=n;i++){
 cout<<i<<endl;
}
```

- 循环嵌套 if 输出 1~n 的偶数:

```
int n;
cin>>n;
for(int i=1;i<=n;i++){
 if(i%2==0){
 cout<<i<<endl;
 }
}
```

- 输出所有的水仙花数:

```
for(int i=100;i<=999;i++){
 int b=i/100; //拆出百位
 int s=i/10%10;// 十位
 int g=i%10; // 个位
 if(b*b*b+s*s*s+g*g*g==i){
 cout<<i<<endl;
 }
}
```

- 反向循环 10~1

```
for(int i=10;i>=1;i--){
 cout<<i<<endl;
}
```

- 循环中输入:

```
int n;
cin>>n;
for(int i=1;i<=n;i++){
 int t;
 cin>>t;
 cout<<"第"<<i<<"个数:"<<t<<endl;
}
```

- n个数的最大最小值:

```
int n;
int maxx=-999; //最大值一开始最小
int maxi; //最大值是第几个
int minn=999; //最小值一开始最大
int mini; //最小值是第几个
cin>>n;
for(int i=1;i<=n;i++){
 int t;
 cin>>t;
 if(t>maxx){ //比最大值还大
 maxx=t; //更新最大值
 maxi=i; //记录最大值的下标
 }
 if(t<minn){
 minn=t;
 mini=i;
 }
}
cout<<"最大值是第"<<maxi<<"个,最大值是:"<<maxx<<endl;
cout<<"最小值是第"<<mini<<"个,最小值是:"<<minn<<endl;
```